

CS 499 Milestone Four Narrative
Giuseppe Claudio Zema
07/29/2026

The artifact selected for this enhancement is the Grazioso Salvare animal rescue management system. The original application was created during my CS 340 Client/Server Development course in C6-2025 (November/December 2025) at SNHU. The original application was a JupyterDash dashboard connected to a MongoDB database containing animal shelter records. Users could search animal records, apply filters, and visualize rescue data through dashboard components.

During previous milestones, this artifact was transformed into a MEAN-stack application consisting of an Angular frontend, Express.js REST API, and MongoDB database. Earlier enhancements focused on improving the software architecture, security, authentication, usability, and algorithmic efficiency of the application. This enhancement continued the evolution of the artifact by focusing specifically on database design, data relationships, security controls, and improved data access patterns.

I selected this artifact because it represents the most complete example of my growth throughout the Computer Science program. The project evolved from a simple academic dashboard into a full-stack application that demonstrates frontend development, backend API design, database modeling, authentication, authorization, testing, and professional software engineering practices.

The original application stored animal records as a single collection and did not support additional relational data management. As the application evolved, additional functionality was required to track animal training history. A database design was needed that could support

multiple training records associated with a single animal while maintaining data consistency and efficient retrieval.

The first database enhancement was the implementation of a one-to-many relationship between `animals` and `training records`. Each animal can now have multiple associated training documents stored in the `training_records` collection. The relationship is maintained through the `animal_id` field, which acts as the reference between the two collections. This design allows training information to grow independently from the animal record while avoiding unnecessary duplication of animal information.

MongoDB schema validation was added to improve data integrity. The Training schema defines required fields, acceptable values, and validation rules for training records. Fields such as `rescue_type`, `training_status`, `training_level`, and `score` include restrictions to prevent invalid data from being stored. This improves reliability by ensuring that records entering the database follow the expected application rules.

The second database enhancement implemented MongoDB aggregation joins using the `$lookup` operator. Previously, training records and animal information were stored separately, requiring additional processing to combine related data. The new aggregation pipeline performs a server-side join between the `training_records` and `animals` collections using the `animal_id` field. The API now returns combined training and animal information, including animal name, breed, and animal type, allowing the frontend to display complete training history without additional client-side processing.

The `$lookup` implementation improved both maintainability and efficiency by moving relationship handling into the database layer. Instead of requiring multiple API calls or manually

combining objects in Angular, MongoDB performs the relationship resolution before returning the response. This creates a cleaner separation of responsibilities between the database, backend API, and frontend application. The trade-off of this approach is that aggregation queries introduce additional database complexity compared to storing all information together, but this cost was acceptable because training history is expected to expand independently from animal records.

Security improvements were also expanded as part of this enhancement. Role-based access control (RBAC) was implemented throughout the application to ensure that users only perform actions appropriate for their permissions. Employee users can view and manage training records, while administrative actions such as deleting records and managing database backups are restricted to administrator accounts.

JWT authentication continues to protect API endpoints by verifying user identity before allowing access to protected resources. Authorization middleware checks user roles before executing database operations, reducing the risk of unauthorized modifications to training information. Audit logging was expanded to track important administrative and training-related operations. Actions such as creating, updating, and deleting training records are recorded with the username, role, timestamp, action type, and additional details. This provides accountability and creates a record of changes made within the system. Audit logs are especially important in applications that manage operational data because they support troubleshooting, security review, and future compliance requirements.

Automated integration tests were updated to verify training CRUD operations, authorization behavior, and the successful retrieval of joined animal and training data.

This enhancement supports Course Outcome Three:

Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices.

The training management enhancement required evaluating database design choices, particularly the trade-off between embedding and referencing related documents in MongoDB. The selected one-to-many relationship using a separate training collection provides better scalability and maintainability while allowing related information to be retrieved efficiently through aggregation.

This enhancement also supports Course Outcome Four:

Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals.

The implementation demonstrates professional database development practices through schema validation, structured data relationships, aggregation queries, and integration between the database, API, and frontend application.

This enhancement also supports Course Outcome Five:

Develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources.

The addition of RBAC, JWT authentication, and audit logging demonstrates a security-focused approach by controlling access to protected operations and maintaining accountability for important system changes.

This enhancement was planned in Module One to address Course Outcomes Three, Four, and Five, and no changes are necessary to that outcome-coverage plan. Previous milestones addressed architecture, authentication, algorithms, and performance improvements, while this enhancement completed the planned database-focused improvements.

Completing this enhancement changed my understanding of database design from simply storing information to designing systems that maintain relationships, security, and long-term maintainability. The original project focused primarily on displaying animal data, but expanding the application required evaluating how new information should be structured, connected, and accessed efficiently.

One of the main challenges was determining the best approach for connecting training records with animal information. MongoDB is a document database, so designing relationships requires evaluating trade-offs between embedding and referencing. While embedding could reduce the complexity of some queries, it would produce larger documents and make it more difficult to manage a growing training history. The referenced one-to-many model combined with the `$lookup` aggregation stage provided a solution that maintained separate collections while still allowing efficient retrieval of related information.

Another challenge was ensuring that database improvements did not introduce security weaknesses. Adding additional functionality also creates additional opportunities for unauthorized access or incorrect data modification. Implementing RBAC, JWT authentication, and audit logging reinforced the importance of designing security controls alongside database functionality rather than adding them afterward.

Overall, this enhancement strengthened my understanding of database engineering principles and demonstrated that professional database solutions require careful consideration of structure, validation, relationships, security, and performance. The completed artifact now better represents a real-world software system capable of supporting future growth while maintaining reliable and secure access to data.